

Distributed Hash Table and the Chord protocol

Assignment 2: Dynamic Membership INF-3200 Distributed Systems Fundamentals

Sindre Frantzen Dalvik
UiT

Tromsø, Norway
sda108@uit.no

Jens Christian Valen Leynse
UiT

Tromsø, Norway
jeley4465@uit.no

Sivert Bottolfsen
UiT

Tromsø, Norway
sbo101@uit.no

Abstract—This implementation extends a peer-to-peer distributed hash table (DHT) based on the Chord protocol to support dynamic membership. The system now allows nodes to join and leave the network dynamically. The addition of HTTP-based join, leave, and simulated crash/recovery functionalities ensures a more robust, scalable, and flexible network structure. The system also remains resilient to node failures, with the network reorganizing itself when nodes become unreachable.

Index Terms—Distributed systems, Chord, Peer-to-peer networks, Dynamic membership, Fault tolerance

I. INTRODUCTION

This assignment enhances a previously implemented distributed key-value store using the Chord protocol by introducing dynamic membership, allowing nodes to join and leave the network. Dynamic membership is essential for managing node churn, which is common in real-world applications like distributed file storage systems. The implementation builds upon the Chord architecture, ensuring each node's successor pointers are updated through a stabilization protocol that runs periodically, enabling efficient lookups even during node joins. When a node joins, it establishes its successor and notifies existing nodes, ensuring reachability is maintained despite concurrent operations.

II. DESIGN

The system is designed to function as a dynamic DHT using the Chord protocol [1]. Each node is responsible for a segment of the keyspace, and nodes communicate using HTTP requests to coordinate the storage and retrieval of key-value pairs.

Key elements of the design include:

- **Node Initialization** Each node is initialized with a unique identifier based on the hash of its ID, which defines its position in the Chord ring. Nodes maintain links to their successor and predecessor to enable efficient lookups and ensure the proper distribution of keys.
- **Dynamic Membership:** This version extends the system to support dynamic node membership through new HTTP API calls. Nodes can join or leave the network, with the ring structure adjusting accordingly to maintain continuity.
 - **Join Process:** A node joins the network by sending a POST request to a specified node in the network, as indicated in the API (POST /join). Upon receiving

this request, the network updates its successor and predecessor pointers to accommodate the new node.

- **Leave Process:** A node can voluntarily leave the network by notifying its neighbors through a POST request (POST /leave). The remaining nodes adjust their successor and predecessor pointers accordingly to maintain the ring's integrity.
- **Simulated Crashes:** The system can simulate node crashes with the POST /sim-crash and POST /sim-recover API calls. When a crash occurs, the affected node stops responding to requests, and the network is expected to handle the failure by adjusting its structure to ensure continued operations.
- **Stabilization and Notification:** A node periodically stabilizes its links to its successor and predecessor to ensure the ring remains intact. The stabilization process was improved by adding the missing *notify* call after *try*, which prevents inconsistencies during the joining process. This ensures that nodes do not mistakenly leave themselves as their own successor or predecessor during stabilization.
- **Consistent Hashing:** The system continues to employ consistent hashing, ensuring minimal disruption when nodes join or leave. Keys are assigned to the closest successor, and when a node leaves, its keys are transferred to its successor.
- **Finger table and efficient lookups:** Each node maintains a finger table that accelerates lookup times, allowing the system to locate nodes in logarithmic time relative to the total number of nodes. This design ensures efficient routing and data retrieval, even as the network grows.
- **Fault tolerance:** By simulating node failures and implementing recovery processes, the system ensures resilience. Nodes can detect when a peer becomes unresponsive and update their routing information to exclude the failed node, improving fault tolerance.
- **Error Handling in Stabilization:** During node crashes or unexpected failures, the system utilizes error-handling logic where the stabilization function attempts to reconnect nodes with their correct predecessors and successors. A failure in the stabilization process is corrected by ensuring nodes send notifications correctly, preventing

potential faults caused by incorrect neighbor references.

A. Advantages

- **Scalability:** Chord's logarithmic lookup time ensures that the system scales efficiently as the number of nodes grows. This property is crucial for large, dynamic networks where nodes frequently join and leave.
- **Fault Tolerance:** The use of successor and predecessor pointers allows the system to tolerate node failures by rerouting requests. Additionally, the system's simulated crash and recovery functions provide insights into how failures are handled and recovered.
- **Load Balancing:** Consistent hashing ensures that keys are evenly distributed among available nodes, preventing any single node from becoming a bottleneck and balancing the load across the network.

B. Disadvantages

- **Join/Leave Overhead:** Each time a node joins or leaves, the network must update its finger tables and reassign key responsibilities. This process incurs a performance cost and introduces a period of instability.
- **Increased Latency:** Network communication between nodes adds latency to GET and PUT requests, especially during periods of high churn. The overhead of maintaining dynamic membership and fault tolerance mechanisms can further exacerbate this issue.
- **Temporary Inconsistencies:** While the system remains operational during node joins and leaves, there can be a short window where key lookups might fail or return stale data until the network stabilizes and keys are reassigned correctly.

III. IMPLEMENTATION

The implementation extends the previous assignment by adding new functionalities and maintaining backward compatibility for key-value storage and retrieval. Key components include:

Node hashing for dynamic membership: Nodes continue to be identified using a hash function, ensuring their placement in the Chord ring. A unique identifier is computed by hashing the node's ID (e.g., IP address), and the node's position is determined by this hash.

HTTP APIs for Dynamic Membership:

- GET /node-info: Returns a node's hash, successor, and known neighbors.
- POST /join: Allows a node to join an existing network.
- POST /leave: Removes the node from the network.
- POST /sim-crash: Simulates a node crash, disabling normal operations.
- POST /sim-recover: Restores the node to an operational state after a simulated crash.

Request Routing: Nodes continue to route requests for key-value storage and retrieval either locally or to other nodes in the network. The finger table is used to speed up the process by skipping over intermediate nodes when appropriate.

Key Storage and Retrieval: When storing a key, the node checks whether the key falls within its responsibility range. If so, the key is stored locally; otherwise, the request is forwarded to the correct node. The retrieval process works similarly, with nodes either returning the value or forwarding the request.

Data consistency: Data consistency is maintained through the Chord protocol's consistent hashing. However, during node joins and leaves, the system does not enforce immediate key redistribution, which may temporarily affect key lookups. Once the network stabilizes, new PUT and GET requests are handled without issue.

IV. EXPERIMENT

Objectives

This experiment aims to evaluate the performance and stability of a distributed key-value store based on the Chord protocol, focusing on the dynamic membership functionality. The specific objectives include measuring the time required for the network to grow and shrink, assessing the network's tolerance to simulated node crashes, and analyzing data availability and request routing during these operations.

A. Time performance analysis

Time to grow network to 32 nodes: Setup: Begin with 32 nodes in a single-node state. Procedure: Use join API calls to incorporate all nodes into the network, measuring the time from the first join call until the network stabilizes at 32 nodes. Methodology: Issue API calls either sequentially or in bursts. Conduct this experiment for network sizes of 2, 4, 8, 16, and 32 nodes, repeating each configuration three times to calculate mean and standard deviation.

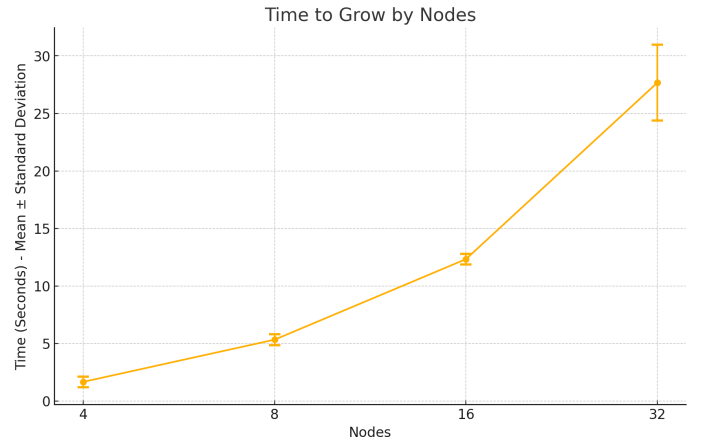


Fig. 1. Data Flow Diagram

Time to shrink network from 32 nodes to 16: Setup: Start with 32 nodes in a stable state. Procedure: Use leave API calls to remove nodes, measuring the time until the network stabilizes at 16 nodes. Methodology: Repeat the shrinking process for sizes of 4, 8, 16, and 32 nodes, reducing the network size by half each time (32 → 16, 16 → 8, etc.). Perform each trial three times for statistical accuracy.

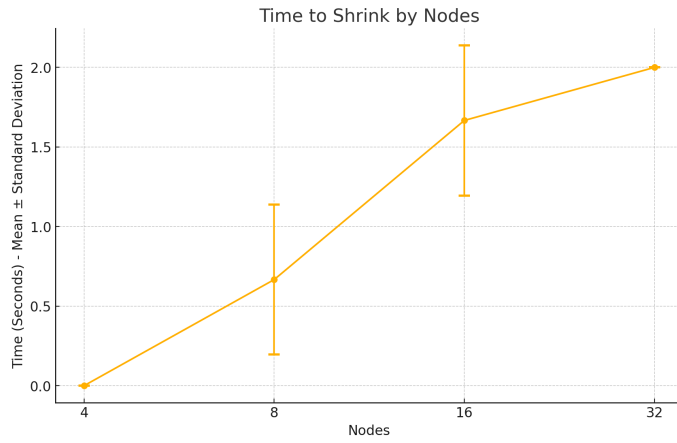


Fig. 2. Data Flow Diagram

B. Crash performance evaluation

Network tolerance to burst of node crashes: Setup: Begin with 32 nodes in a stable network. Procedure: Utilize the simulate-crash API call to "crash" nodes, initially testing one node, then progressing to bursts of two, three, and more. Methodology: Measure the time taken for the network to stabilize again after each crash scenario. Determine the maximum number of simultaneous crashes the network can tolerate while maintaining stability. Results: After running 4 tests with crashing several nodes. The highest the network was able to stabilize from was 75% at 10 crashed nodes and 25% at 11 crashed nodes (The node network couldn't manage to stabilize with more losses at the same time, but worked fine with several smaller ones at the same scale). Since the nodes removed were chosen on random, the performance varied. The best time it managed at 10 crashes was 3 seconds to stabilize, and otherwise it could take 15-20 seconds on average.

Stability Probing: Develop code to probe the network's stability and define when it can be considered stable, using a reliable timer mechanism to measure the duration of operations accurately.

V. DISCUSSION

We followed the implementation described in the chord paper quite closely. Only in a few instances did we deviate. For example in a edge case when the first process joins, where we had to implement a edge case fix for updating the pointers. Another example is our interpretation of the successor, where we do a search in the finger table for a responding successor (as opposed to a solution of multiple successors in a list, described in the chord paper).

Also figure out when nodes had crashed we had the node respond with 404 instead of implementing timeout.

VI. CONCLUSION

Overall the implementation works as expected, with only some few unwanted behaviors, such as troubles stabilizing the network when too many nodes crash.

REFERENCES

- [1] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup protocol for internet applications," *IEEE/ACM Transactions on Networking*, vol. 11, no. 1, pp. 17–32, 2003.
- [2] M. v. Steen and A. S. Tanenbaum, *Distributed Systems: Principles and Paradigms*, 4th ed., Pearson Education, 2023.